hashlock.

Security Audit

Prophet (DeFi)

April 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

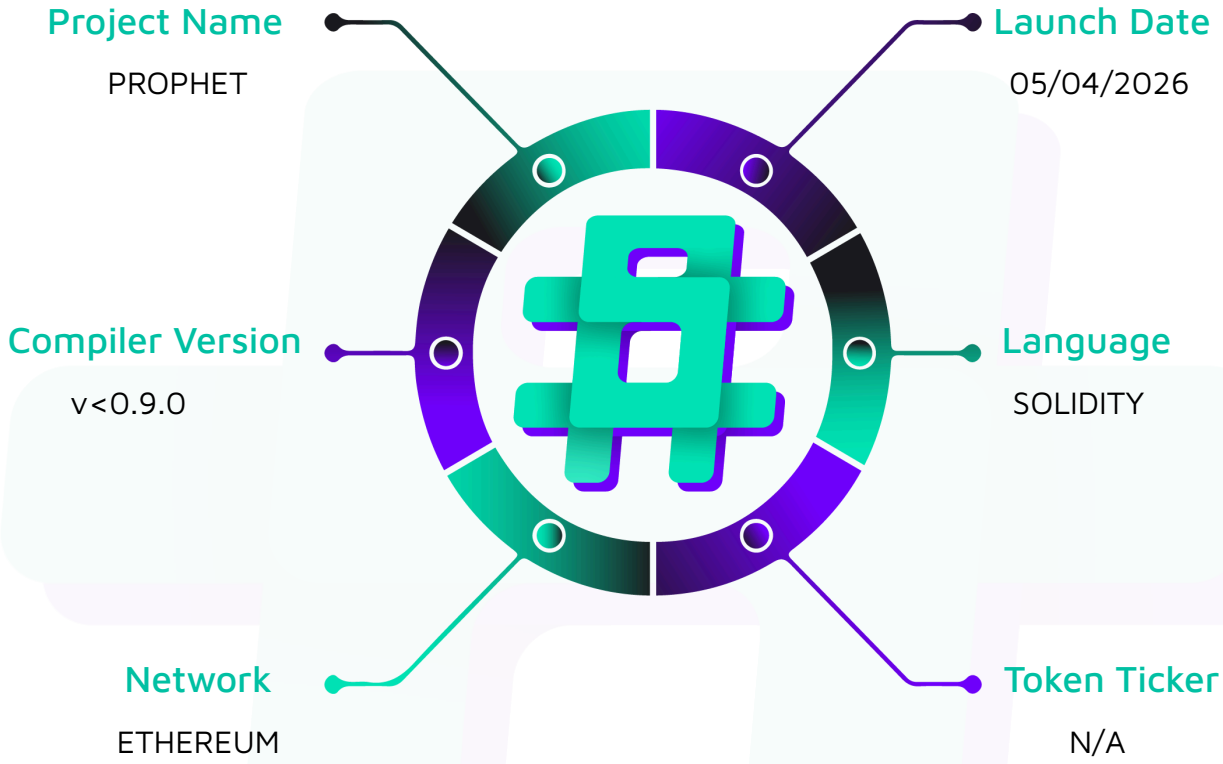
The Prophet team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

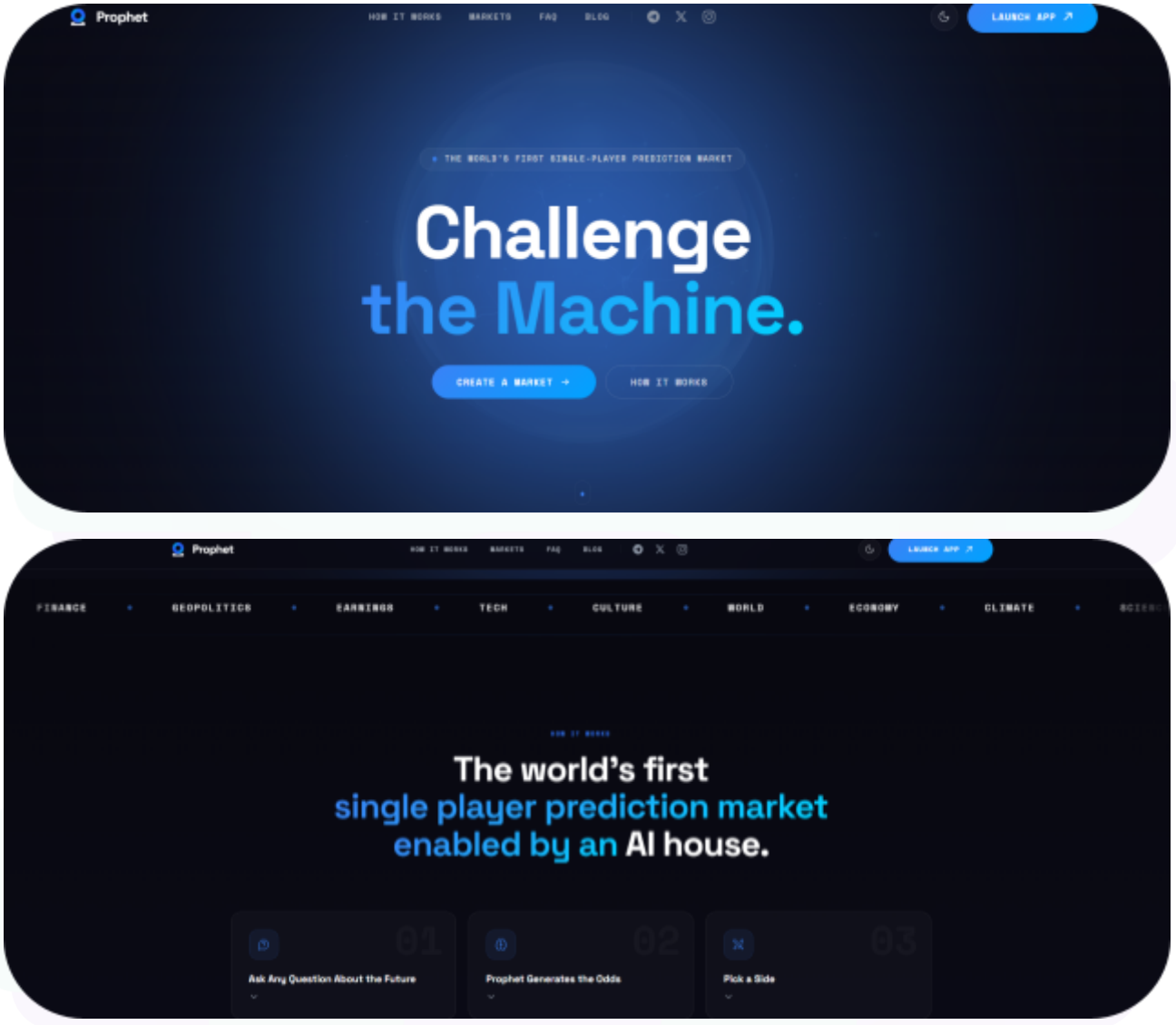
Prophet is a decentralized prediction market built on the Obyte network, allowing users to speculate on future events by purchasing outcome tokens (e.g., "YES" or "NO"). Prices are determined algorithmically via bonding curves, enabling continuous liquidity without traditional order books. Users can participate as traders or liquidity providers, earning fees or profiting from correct predictions. Upon resolution, winning tokens are redeemable while losing tokens become worthless. The system's security depends on reliable outcome resolution, sound economic design, and resistance to manipulation.

Project Name: Prophet
Project Type: DeFi
Compiler Version: <0.9.0
Website: prophetmarket.ai
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the Solidity code within the Prophet project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Prophet Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**April 2026**
Contract 1	ProphetCTFExchange.sol
Contract 2	Resolution.sol
Contract 3	PolySafeLib.sol
Contract 4	Deploy.s.sol
Contract 5	AddOperator.s.sol
Contract 6	PolySafeProxyFactory.sol
Contract 7	Deps.sol
Contract 8	DeployPolySafeFactory.s.sol
Audited GitHub Commit Hash 1	752f22d24e6785d655cfd6f7f054cff8a905f251
Audited GitHub Commit Hash 2	a83088e0015fd4e0859665b654648f28ef28c177
Audited GitHub Commit Hash 3	d1ad8002371bfc0ea4b06e310e0e490dba544907
Fix Review GitHub Commit Hash 1	968f1b60b89d53eda413258761970a8301e65beb

Fix Review GitHub Commit Hash 2	bb302fa83db04db7674b44388fe03683f267cc9c
Fix Review GitHub Commit Hash 3	3de5d2ddb5d9980087970408e1b1927c64a6d624

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

6 High severity vulnerabilities

11 Medium severity vulnerabilities

5 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ProphetCTFExchange.sol - Allows the admin to: - Pause / unpause trading - Set the proxy factory address - Set the safe factory address - Set the oracle address - Unregister a token pair (escape hatch for bad registrations) - Allows the admin or oracle to: - Register a YES/NO token pair against a CTF conditionId (validated on-chain) - Allows the operator to: - Fill a single signed order - Fill multiple signed orders in a batch - Match a taker order against maker orders - Allows users to: - Sign EIP-712 orders (EOA / POLY_PROXY / POLY_GNOSIS_SAFE / POLY_1271) for operators to fill - Cancel their own orders - Increment their own nonce to invalidate outstanding orders	**Contract achieves this functionality.**
Resolution.sol - Allows the owner (admin) to: - Transfer the oracle role via 2-step Ownable2Step - renounceOwnership is disabled to prevent bricking oracle management	**Contract achieves this functionality.**

- Allows the oracle to: - Report payouts exactly once per conditionId - Submit one of three valid payout vectors: [1,0] (YES), [0,1] (NO), [1,1] (cancelled 50/50) - Attach an IPFS CID with resolution reasoning - Allows users to: - Read recorded payouts for any conditional Check whether a conditionId has been reported - Read the IPFS CID for any conditionId	
PolySafeProxyFactory.sol Allows users to: - Create a 1-of-1 Gnosis Safe proxy on behalf of an EOA owner using a valid EIP-712 signature over (paymentToken, payment, paymentReceiver, nonce, deadline) - Compute the deterministic CREATE2 proxy address for any owner - Read proxyCreationCode, getContractBytecode, per-signer nonces, and the current domainSeparator Hardened per prior audit: - Per-signer nonce makes signatures non-replayable and revocable - Signatures carry a deadline (auto-expire) - Domain separator is recomputed if block.chainid changes (fork-safe)	**Contract achieves this functionality.**
Deps.sol - Force-compile helper only:	**Contract achieves this functionality.**

- Imports CompatibilityFallbackHandler and GnosisSafeL2 so Foundry produces their artifacts for deployment. Abstract contract with no state, no functions, no external surface.	

Code Quality

This audit scope involves the smart contracts of the Prophet project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Prophet project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Resolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] ProphetCTFExchange#unregisterToken — unchecked complement enables arbitrary registry deletion and stale-order reactivation

Description

`unregisterToken(uint256 token, uint256 complement)` accepts a fully attacker-controlled `complement` parameter and deletes `registry[complement]` without verifying it equals `registry[token].complement`.

Vulnerability Details

```
// ProphetCTFExchange.sol:128
function unregisterToken(uint256 token, uint256 complement) external onlyAdmin {
    bytes32 conditionId = registry[token].conditionId;
    delete registry[token];
    delete registry[complement]; // never validated against
registry[token].complement
    emit TokenUnregistered(token, complement, conditionId);
}
```

Two consequences:

1. Admin can pass any unrelated `tokenX` as `complement` and delete that unrelated market's registration. `validateTokenId(tokenX)` then reverts with `InvalidTokenId` and the market is bricked. Re-registration is blocked because the sibling of `tokenX` still has a non-zero `complement` pointer, so `Registry._registerToken` reverts with `AlreadyRegistered`.
2. After a legitimate unregister + re-register of the same pair, the stored `orderStatus` entries for that pair are not cleared. Previously-signed orders — including near-filled ones — become fillable again at stale signed prices.

Proof of Concept

```
// State: market A registered (tokenA0, tokenA1), market B registered (tokenB0,
tokenB1).
exchange.unregisterToken(tokenA0, tokenB0);
// registry[tokenA0] and registry[tokenB0] are both deleted.
// Market B can no longer trade — validateTokenId(tokenB0) reverts.
// Market B cannot be re-registered because registry[tokenB1].complement != 0.
```

Impact

Any live market can be permanently bricked by a single admin call. Combined with re-register of the same pair, previously signed orders at outdated prices resume filling — permitting colluding admin + operator to extract the delta between signed and current prices from makers.

Recommendation

Remove the `complement` parameter and read it from storage. Additionally, introduce a per-pair `registrationEpoch` that orders must commit to at signing time, so orders signed under a prior registration cannot be revived.

Status

Resolved

[H-02] Resolution#reportPayouts — never forwards to ConditionalTokens, leaving redemption dependent on an off-chain keeper

Description

`Resolution.reportPayouts` persists payout data in its own storage but never calls `ConditionalTokens.reportPayouts`. Without that call, `payoutDenominator(conditionId)` remains `0` in the CTF and `redeemPositions` is permanently blocked for all holders.

Vulnerability Details

```
// Resolution.sol:64
function reportPayouts(bytes32 conditionId, uint256[] calldata payouts, string
calldata ipfsCid)
    external onlyOracle
{
    if (_reported[conditionId]) revert AlreadyReported(conditionId);
    if (payouts.length != 2) revert InvalidPayoutsLength(payouts.length);
    if (!_validPayouts(payouts[0], payouts[1])) revert InvalidPayoutValues();

    _reported[conditionId] = true;
    _payouts[conditionId] = payouts;
    _ipfsCids[conditionId] = ipfsCid;
    emit PayoutsReported(conditionId, payouts, ipfsCid);
    // NO call to IConditionalTokens(ctf).reportPayouts(...)
}
```

Only an off-chain keeper can bridge Prophet's record with the CTF, and that keeper is a single unaudited point of failure.

Proof of Concept

```
// Oracle resolves in Prophet but the keeper is down / disabled:
resolution.reportPayouts(conditionId, [1,0], "ipfs://...");
ctf.redeemPositions(...); // reverts — payoutDenominator still 0
```

Impact

If the keeper fails, is compromised, or is withheld, user collateral bound to winning CTF positions is permanently locked on-chain with no recovery path. A malicious oracle can exploit this by reporting to Prophet while ensuring `CTF.reportPayouts` is never called.

Recommendation

Either call `IConditionalTokens(ctf).reportPayouts(questionId, payouts)` atomically from `Resolution.reportPayouts`, or add a callable function that any address can invoke post-dispute-window to forward the stored values to the CTF.

Status

Resolved

[H-03] PolySafeLib — hardcoded proxyCreationCode bytecode diverges from factory, silently breaking all Safe signature validation

Description

`PolySafeLib.proxyCreationCode` is a 369-byte compile-time constant (including IPFS metadata hash) used for CREATE2 address derivation, while the in-repo `PolySafeProxyFactory` derives addresses from runtime `type(GnosisSafeProxy).creationCode`. Any mismatch between the hardcoded constant and the factory's actual deployed proxy bytecode causes every `POLY_GNOSIS_SAFE` signature verification to return `false`.

Vulnerability Details

```
// PolySafeLib.sol:10
bytes private constant proxyCreationCode =
    hex"608060405234801561001057600080fd...a264697066735822...0033"; // includes IPFS
hash

// PolySafeProxyFactory.sol:76-78
function proxyCreationCode() public pure returns (bytes memory) {
    return type(GnosisSafeProxy).creationCode; // recomputed at compile time
}
```

- The library is `pragma solidity <0.9.0;` the factory is `pragma solidity >=0.7.0 <0.9.0;`. Compiling the two with different solc/optimizer/EVM-target combinations produces different metadata hashes → different keccak256 → different CREATE2 addresses.
- The in-source comment confirms the constant has already drifted once (Hardhat → Foundry migration).

Proof of Concept

```
address libDerived   = PolySafeLib.getSafeAddress(user, masterCopy, factory);
address factoryDerived = PolySafeProxyFactory(factory).computeProxyAddress(user);
// libDerived != factoryDerived if the hardcoded bytecode differs from the
// factory&#x27;s runtime bytecode -> verifyPolySafeSignature returns false for all
```

```
sigs.
```

Impact

Complete DoS of the `POLY_GNOSIS_SAFE` signature type. All safe-based orders silently fail signature validation, locking users out of trading until redeployment.

Recommendation

Replace the hardcoded constant with a runtime call to the factory's `proxyCreationCode()` / `getContractBytecode()` so the address derivation always matches the live bytecode. Alternatively, add a deployment-time assertion that fails if `keccak256(PolySafeLib bytecode) != keccak256(factory.getContractBytecode())`.

Status

Resolved

[H-04] Trading#_updateTakingWithSurplus — taker payout is inflated by direct ERC-1155/ERC-20 donations to the exchange

Description

In `_matchOrders`, the taker's final `taking` amount is determined by the exchange's raw `balanceOf(address(this))` after the maker fills, not by the delta between before and after. Any tokens that happen to sit at the exchange (including tokens donated by an attacker front-running the `matchOrders` call) are transferred out to the taker.

Vulnerability Details

```
// Trading.sol:149
taking = _updateTakingWithSurplus(taking, takerAssetId);

// Trading.sol:371
function _updateTakingWithSurplus(uint256 minimumAmount, uint256 tokenId) internal
returns (uint256) {
    uint256 actualAmount = _getBalance(tokenId); // raw balanceOf(address(this))
    if (actualAmount < minimumAmount) revert TooLittleTokensReceived();
    return actualAmount;
}

// AssetOperations.sol:18
function _getBalance(uint256 tokenId) internal override returns (uint256) {
    if (tokenId == 0) return IERC20(getCollateral()).balanceOf(address(this));
    return IERC1155(getCtf()).balanceOf(address(this), tokenId);
}
```

Fees are computed on the inflated `taking` amount — amplifying the operator's extraction as well.

Proof of Concept

```
// Attacker front-runs matchOrders on a public mempool chain:
IERC1155(ctf).safeTransferFrom(attacker, address(exchange), takerAssetId,
donatedAmount, "");
// Operator&#x27;s matchOrders tx lands next:
```

```
exchange.matchOrders(takerOrder, makerOrders, fill, fills);
// taker receives (normal taking + donatedAmount) - fee-on-inflated-amount
```

Impact

Direct value extraction from donated tokens into the taker's account (and, through inflated fees, into the operator's account). On public-mempool chains the donation front-run is permissionless; on private-mempool chains a colluding operator can stage the donation directly.

Recommendation

Replace the balance-based surplus with delta-based accounting:

```
uint256 balanceBefore = _getBalance(takerAssetId);
_fillMakerOrders(takerOrder, makerOrders, makerFillAmounts);
uint256 taking = _getBalance(takerAssetId) - balanceBefore;
if (taking < minimumTaking) revert TooLittleTokensReceived();
```

Apply the same pattern to the `refund = _getBalance(makerAssetId)` logic at `Trading.sol:163`, which has the identical donation-exposure flaw for `makerAssetId`.

Status

Resolved

[H-05] ProphetCTFExchange#_validateTokenCondition — only verifies math, not who prepared the condition in the CTF

Description

`_validateTokenCondition` checks the pure mathematical relationship between `tokenId`, `complement`, and `conditionId` using `getCollectionId` and `getPositionId`, but never verifies that (a) the condition has been prepared in the CTF at all, or (b) which oracle is registered with the CTF for that condition.

Vulnerability Details

```
// ProphetCTFExchange.sol:140
function _validateTokenCondition(uint256 token, uint256 complement, bytes32
conditionId) internal view {
    IConditionalTokens ctfContract = IConditionalTokens(ctf);
    IERC20 collateralToken = IERC20(collateral);

    bytes32 collectionYes = ctfContract.getCollectionId(bytes32(0), conditionId, 1);
    bytes32 collectionNo  = ctfContract.getCollectionId(bytes32(0), conditionId, 2);

    uint256 expectedYes = ctfContract.getPositionId(collateralToken, collectionYes);
    uint256 expectedNo  = ctfContract.getPositionId(collateralToken, collectionNo);

    bool valid = (token == expectedYes && complement == expectedNo)
              || (token == expectedNo  && complement == expectedYes);
    if (!valid) revert TokenConditionMismatch();
}
```

Both CTF helpers are pure functions — they return deterministic hashes regardless of whether `prepareCondition` was ever called. The NatSpec claim that they "will revert" for unprepared conditions is incorrect.

Proof of Concept

```
// Compromised oracle chooses itself as the CTF-level oracle:
ctf.prepareCondition(compromisedOracle, questionId, 2);
bytes32 conditionId = ctf.getConditionId(compromisedOracle, questionId, 2);
```

```
// Exchange check passes — it&#x27;s pure math:
exchange.registerToken(expectedYes, expectedNo, conditionId);

// Users trade and accumulate positions. Oracle then resolves UNILATERALLY at the CTF
// layer, bypassing Resolution.sol completely:
ctf.reportPayouts(questionId, [1, 0]);
ctf.redeemPositions(collateral, 0, conditionId, [1]);
// Compromised oracle drains the market.
```

Impact

A compromised oracle (or any operator given the oracle role) can register markets whose CTF-level oracle is attacker-controlled and extract full TVL by calling `CTF.reportPayouts` directly — bypassing Resolution's record, the IPFS audit trail, and any future dispute window.

Recommendation

In `_validateTokenCondition`, additionally require `ctfContract.getOutcomeSlotCount(conditionId) == 2`. Then, if the CTF version permits, verify the condition's registered oracle address equals an admin-configured expected oracle (or restrict the set of allowed `questionId` preimages via a whitelist).

Status

Resolved

[H-06] Trading#_matchOrders — refund uses raw exchange balance of makerAssetId, donatable the same way as taking

Description

After filling maker orders, `_matchOrders` refunds any remaining `makerAssetId` balance at the exchange back to the taker's maker address using the raw `balanceOf` read. This has the identical donation-extraction shape as H-04 but for the maker-side asset.

Vulnerability Details

```
// Trading.sol:163
uint256 refund = _getBalance(makerAssetId);
if (refund > 0) _transfer(address(this), takerOrder.maker, makerAssetId, refund);
```

Any `makerAssetId` balance that happens to be at the exchange at this moment — including tokens just donated by an attacker — is transferred to `takerOrder.maker`. For a BUY taker, `makerAssetId == 0` (collateral), so donated collateral is siphoned into the taker. For a SELL taker, donated CTF outcome tokens are siphoned.

Proof of Concept

```
// Attacker front-runs a known matchOrders for a BUY taker:
IERC20(collateral).transfer(address(exchange), donatedUSDC);
// Operator executes matchOrders.
// takerOrder.maker receives donatedUSDC as a "refund" even though nothing was
overpulled.
```

Impact

Direct value extraction — attacker-donated tokens are transferred to the taker order's maker, which a sophisticated attacker can choose by timing the donation against a particular taker order they control.

Recommendation

Replace the raw read with a delta, e.g.:

```
uint256 pulledIn = making; // tokens pulled from taker.maker into exchange
uint256 usedUp   = takingInCounterparty; // what mint/merge consumed on that side
uint256 refund   = pulledIn > usedUp ? pulledIn - usedUp : 0;
```

Or snapshot `balanceOf` at function entry and refund only `current - entrySnapshot` that logically belongs to the taker.

Status

Resolved

Medium

[M-01] CalculatorHelper#isCrossing — treats orders with takerAmount == 0 as universally crossing, enabling free token drain

Description

`isCrossing` unconditionally returns `true` when either side has `takerAmount == 0`, and `_validateOrder` contains no check that rejects `takerAmount == 0`. Such an order quotes a "price" of zero consideration.

Vulnerability Details

```
// CalculatorHelper.sol:63
function isCrossing(Order memory a, Order memory b) internal pure returns (bool) {
    if (a.takerAmount == 0 || b.takerAmount == 0) return true;
    ...
}

// CalculatorHelper.sol:11
function calculateTakingAmount(uint256 makingAmount, uint256 makerAmount, uint256
takerAmount)
    internal pure returns (uint256)
{
    if (makerAmount == 0) return 0;
    return makingAmount * takerAmount / makerAmount; // = 0 when takerAmount == 0
}
```

The maker transfers their full `makerAmount` and receives zero in return.

Proof of Concept

```
Order memory bad = Order({..., makerAmount: 1_000 ether, takerAmount: 0, ...});
exchange.fillOrder(bad, 1_000 ether); // maker loses 1000 tokens, receives 0
```

Impact

A maker who signs an order with `takerAmount = 0` (malicious front-end, tampered calldata, buggy client) donates the entire signed position for zero consideration. A malicious operator is required to submit it but is always the submitter in this protocol.

Recommendation

```
if (order.makerAmount == 0 || order.takerAmount == 0) revert InvalidAmount();
```

in `_validateOrder`.

Status

Resolved

[M-02] Auth — no minimum-admin guard; last admin can permanently lock the contract

Description

`removeAdmin` and `renounceAdminRole` unconditionally clear `admins[addr]`. No admin count is tracked, and there is no guard preventing removal of the final admin.

Vulnerability Details

```
// Auth.sol:57
function removeAdmin(address admin) external onlyAdmin {
    admins[admin] = 0;
    emit RemovedAdmin(admin, msg.sender);
}

function renounceAdminRole() external onlyAdmin {
    admins[msg.sender] = 0;
    emit RemovedAdmin(msg.sender, msg.sender);
}
```

Proof of Concept

```
auth.renounceAdminRole(); // zero remaining admins
auth.addAdmin(newAdmin);  // reverts: onlyAdmin
```

Impact

Once the final admin is removed, `addAdmin`, `addOperator`, `pauseTrading`, `unpauseTrading`, `setOracle`, `setProxyFactory`, `setSafeFactory`, and `unregisterToken` become permanently inaccessible. The exchange continues matching trades on whatever state it is stuck in, with no ability to recover.

Recommendation

Track an `adminCount` and revert `removeAdmin` / `renounceAdminRole` when `adminCount == 1`. Alternatively, migrate to a two-step transfer pattern that preserves at least one reachable admin.

Status

Resolved

[M-03] Trading#_matchOrders — taker fee computed on surplus-inflated taking, exceeding signed fee expectations

Description

After `_updateTakingWithSurplus` potentially raises `taking` above what the taker's limit implied, the fee is computed on this inflated value. Users consent only to a `feeRateBps` against their own limit price, not against any later surplus.

Vulnerability Details

```
// Trading.sol:149
taking = _updateTakingWithSurplus(taking, takerAssetId);
uint256 fee = CalculatorHelper.calculateFee(
    takerOrder.feeRateBps,
    takerOrder.side == Side.BUY ? taking : making,
    making, taking,
    takerOrder.side
);
```

`CalculatorHelper.calculateFee` uses the (inflated) `taking` both as the outcome-token base and as input to `_calculatePrice`, so the fee scales super-linearly when `taking` rises above the signed quote.

Impact

The actual fee paid by a taker can silently exceed the fee implied by the signed order. Combined with H-04 this is a direct operator-extraction amplifier.

Recommendation

Compute the taker fee against the originally signed `takerAmount` (scaled by `makingAmount / makerAmount`), not against the post-surplus amount.

Status

Resolved

[M-04] CalculatorHelper#calculateFee / calculateTakingAmount — integer truncation to zero enables fillAmount=1 micro-extraction

Description

Both the taking amount and the fee use unsafe integer division. For small `fillAmount` at skewed prices, both round to zero — the maker transfers tokens while the operator pays nothing in consideration and nothing in fees.

Vulnerability Details

```
// CalculatorHelper.sol:17
return makingAmount * takerAmount / makerAmount;

// CalculatorHelper.sol:40
fee = (feeRateBps * min(price, ONE - price) * outcomeTokens) / (price * BPS_DIVISOR);
```

At `fillAmount = 1` on an order where `takerAmount / makerAmount < 1`, `takingAmount == 0` and the fee is `0`.

Impact

On cheap L2s, millions of `fillAmount=1` fills are economically viable. A malicious operator can slowly drain the maker's `makerAmount` without paying for the taker side.

Recommendation

Add `if (takingAmount == 0) revert RoundsToZero();` (or equivalent) inside `_performOrderChecks`. Additionally consider a minimum fee floor of at least `1` wei when `feeRateBps > 0`.

Status

Resolved

[M-05] Trading#_matchOrders — operator fees charged in the losing outcome token create unbacked, correlated directional exposure

Description

For a BUY taker, `takerAssetId` is a CTF outcome token (YES or NO), not collateral. The fee transfer therefore pays the operator in ERC-1155 outcome tokens.

Vulnerability Details

```
// Trading.sol:160
_chargeFee(address(this), msg.sender, takerAssetId, fee);
// takerAssetId = CTF tokenId for BUY takers
```

`_fillFacingExchange` does the same at `Trading.sol:272`. If the market later resolves against that side, those fee tokens are worth zero.

Impact

Operator fee revenue inherits market-directional risk. Over many markets, correlated negative resolutions can wipe out significant accumulated operator balances with no offset. There is no mechanism to merge complementary YES+NO fee tokens into collateral.

Recommendation

Either charge the fee in collateral regardless of the order side, or periodically sweep complementary outcome tokens via `mergePositions` on `min(balance(yes), balance(no))` and retain only the collateral proceeds.

Status

Resolved

[M-06] ProphetCTFExchange#registerToken — missing notPaused, allowing new markets to go live on unpause with no review window

Description

`pauseTrading` gates `fillOrder`, `fillOrders`, and `matchOrders`, but not `registerToken`. A compromised oracle can register new (potentially malicious) token pairs during an active pause; on unpause the new markets are live immediately.

Vulnerability Details

```
// ProphetCTFExchange.sol:117 — no notPaused modifier
function registerToken(uint256 token, uint256 complement, bytes32 conditionId)
external {
    if (admins[msg.sender] != 1 && msg.sender != oracle) revert NotAdminOrOracle();
    _validateTokenCondition(token, complement, conditionId);
    _registerToken(token, complement, conditionId);
}
```

Impact

Pausing no longer acts as a safety circuit for registration — the very action the pause is usually intended to halt remains executable.

Recommendation

Add the `notPaused` modifier to `registerToken`.

Status

Resolved

[M-07] Resolution — no pause, dispute window, or admin override on reportPayouts

Description

`reportPayouts` is write-once per `conditionId`. There is no pause, no dispute window, and no admin override. A compromised oracle can corrupt every unreported market in a single block with no on-chain recovery.

Vulnerability Details

```
// Resolution.sol:64
function reportPayouts(...) external onlyOracle {
    if (_reported[conditionId]) revert AlreadyReported(conditionId);
    _reported[conditionId] = true;
    _payouts[conditionId]   = payouts;
    _ipfsCids[conditionId]  = ipfsCid;
}
```

The only mitigation is `Ownable2Step` ownership transfer, which takes at least one block and requires the owner to be actively monitoring.

Recommendation

Add `Pausable` (OpenZeppelin) to gate `reportPayouts`, and consider a short (e.g. 24h) dispute window before payouts are considered final and redeemable downstream.

Status

Resolved

[M-08] PolySafeProxyFactory#createProxy — missing zero-address guard after ECDSA recovery

Description

`createProxy` uses the recovered signer directly as `owner` without a zero-address check. For older OpenZeppelin ECDSA releases (prior to 4.8.x hardening), malformed signatures can recover to `address(0)`, and `nonces[address(0)] == 0` at initialization, so the nonce check trivially passes.

Vulnerability Details

```
// PolySafeProxyFactory.sol:106
address owner = _getSigner(paymentToken, payment, paymentReceiver, nonce, deadline,
createSig);
require(nonces[owner]++ == nonce, "SafeProxyFactory: invalid nonce");
_deploySafeFor(owner, paymentToken, payment, paymentReceiver);
```

Library versioning matters here — the module is declared as a submodule without pinning, so the recovered-zero-address path cannot be ruled out at audit time.

Impact

A Safe is deployed with `address(0)` as sole owner at a deterministic CREATE2 slot, occupying that slot permanently. In the worst case, the attacker mints a Safe whose owner is `address(0)` at the victim's expected address.

Recommendation

```
address owner = _getSigner(...);
require(owner != address(0), "SafeProxyFactory: invalid signature");
require(nonces[owner]++ == nonce, "SafeProxyFactory: invalid nonce");
```

Status

Resolved

[M-09] PolyFactoryHelper — missing zero-address checks on _setProxyFactory and _setSafeFactory

Description

`setProxyFactory(address(0))` or `setSafeFactory(address(0))` permanently break the corresponding signature type (all `POLY_PROXY` or `POLY_GNOSIS_SAFE` validations) until an admin calls the setter again with a valid factory.

Vulnerability Details

```
// PolyFactoryHelper.sol:62
function _setProxyFactory(address _proxyFactory) internal {
    emit ProxyFactoryUpdated(proxyFactory, _proxyFactory);
    proxyFactory = _proxyFactory;
}

function _setSafeFactory(address _safeFactory) internal {
    emit SafeFactoryUpdated(safeFactory, _safeFactory);
    safeFactory = _safeFactory;
}
```

`setOracle` checks non-zero; these setters do not.

Recommendation

```
function _setProxyFactory(address _proxyFactory) internal {
    require(_proxyFactory != address(0), "zero address");
    ...
}
```

Status

Resolved

[M-10] ProphetCTFExchange constructor — no zero-address validation on immutable collateral / ctf

Description

The constructor forwards `_collateral` and `_ctf` to immutable storage in the `Assets` mixin without zero-address checks. Because the fields are immutable, a deployment with bad inputs cannot be corrected — the exchange is permanently bricked.

Vulnerability Details

```
// ProphetCTFExchange.sol:53
constructor(address _collateral, address _ctf, address _proxyFactory, address
_safeFactory)
    Assets(_collateral, _ctf)
    Signatures(_proxyFactory, _safeFactory)
{}

// Assets.sol:12 — no non-zero checks
constructor(address _collateral, address _ctf) {
    collateral = _collateral;
    ctf = _ctf;
    IERC20(collateral).approve(ctf, type(uint256).max);
}
```

Recommendation

```
constructor(address _collateral, address _ctf, address _proxyFactory, address
_safeFactory) ... {
    require(_collateral != address(0) && _ctf != address(0), "zero address");
}
```

Status

Resolved

[M-11] GnosisSafe.setup call — paymentReceiver == address(0) routes payment to tx.origin

Description

When `payment > 0` and `paymentReceiver == address(0)`, Gnosis Safe's `setup()` implementation falls back to `tx.origin` as the payout recipient. An attacker who sees the signed `createProxy` calldata in the mempool can resubmit it under their own EOA, becoming `tx.origin` and capturing the payment.

Vulnerability Details

```
// PolySafeProxyFactory.sol:128
proxy.setup(owners, 1, address(0), "", fallbackHandler, paymentToken, payment,
paymentReceiver);
```

The signature covers `paymentReceiver`, so the signer must cooperate with `paymentReceiver == address(0)` for this to trigger — but nothing in `createProxy` blocks the combination.

Impact

A signer can be tricked (or default to) setting `paymentReceiver = 0`, and any observer on a public mempool can steal the payment by front-running the submission.

Recommendation

```
require(payment == 0 || paymentReceiver != address(0),
    "SafeProxyFactory: zero receiver with non-zero payment");
```

Status

Resolved

Low

[L-01] PolyFactoryHelper — factory change retroactively invalidates all pending signature-bound orders

Description

`setProxyFactory` and `setSafeFactory` take effect instantly. All existing `POLY_PROXY` / `POLY_GNOSIS_SAFE` orders become unfillable with no grace period and no user-facing cancellation mechanism.

Vulnerability Details

```
// PolyFactoryHelper.sol:62
function _setProxyFactory(address _proxyFactory) internal {
    emit ProxyFactoryUpdated(proxyFactory, _proxyFactory);
    proxyFactory = _proxyFactory;
}
```

Signature verification for `POLY_PROXY` and `POLY_GNOSIS_SAFE` derives the wallet address from the current `proxyFactory` / `safeFactory`; orders signed under the prior factory no longer match and revert with `InvalidSignature`.

Proof of Concept

```
// User signs a POLY_PROXY order bound to the current proxyFactory.
exchange.setProxyFactory(newProxyFactory);
// The user&#x27;s order now fails signature validation on the next fill attempt.
```

Impact

All pending orders that rely on proxy-wallet or safe-wallet signature derivation silently become unfillable the moment admin changes a factory. Users cannot cancel on-chain without knowing about the change.

Recommendation

Add a timelock on factory changes or version-tag orders so that orders signed under the old factory remain valid against the old factory.

Status

Resolved

[L-02] Trading#fillOrders / matchOrders — no array-length validation

Description

Both batch entry points iterate over companion arrays without first verifying that the arrays have equal length. A mismatched length produces an opaque out-of-bounds revert.

Vulnerability Details

```
// Trading.sol:115
function _fillOrders(Order[] memory orders, uint256[] memory fillAmounts, address to)
internal {
    uint256 length = orders.length;
    uint256 i = 0;
    for (; i < length;) {
        _fillOrder(orders[i], fillAmounts[i], to);
        ...
    }
}
```

If `fillAmounts.length < orders.length`, the loop reverts mid-iteration on `fillAmounts[i]` with an unhelpful panic code.

Proof of Concept

```
Order[] memory orders = new Order[](3);
uint256[] memory fills = new uint256[](2);
exchange.fillOrders(orders, fills); // panic (0x32), not ArrayLengthMismatch
```

Impact

Operator-side debugging and off-chain tooling cannot distinguish a length mismatch from unrelated out-of-bounds conditions.

Recommendation

Add explicit length checks:

```
if (orders.length != fillAmounts.length) revert ArrayLengthMismatch();
```

Status

Resolved

[L-03] Hashing — public domainSeparator getter returns a stale immutable after a chain fork

Description

`draft-EIP712`'s internal `_domainSeparatorV4()` recomputes when `block.chainid` changes, so the actual order signature verification is fork-safe. However, the publicly exposed `domainSeparator` is stored as an `immutable` set once at construction, so off-chain consumers reading the getter on a forked chain receive stale data.

Vulnerability Details

```
// Hashing.sol:11-15
bytes32 public immutable domainSeparator;

constructor(string memory name, string memory version) EIP712(name, version) {
    domainSeparator = _domainSeparatorV4(); // frozen at deploy-time chainid
}
```

Proof of Concept

```
// On a forked chain, off-chain signer reads the public getter:
bytes32 bad = exchange.domainSeparator(); // returns the pre-fork value
// signatures built against `bad` will not verify on-chain
```

Impact

Off-chain signers using the public getter after a fork produce signatures that do not verify. On-chain verification continues to work because `_hashTypedDataV4` uses the live value.

Recommendation

Replace the immutable with a view function delegating to `_domainSeparatorV4()`, or migrate from `draft-EIP712` to non-draft OpenZeppelin `EIP712`.

Status

Resolved

[L-04] ProphetCTFExchange#_validateTokenCondition — NatSpec is misleading; non-binary conditions are not explicitly rejected

Description

The inline comment claims validation will revert when a condition has not been prepared. That is false — `IConditionalTokens.getCollectionId` and `getPositionId` are pure functions and return deterministic hashes regardless of whether `prepareCondition` was ever called. Additionally, conditions with `outcomeSlotCount != 2` are not explicitly rejected.

Vulnerability Details

```
// ProphetCTFExchange.sol:140-154
function _validateTokenCondition(uint256 token, uint256 complement, bytes32
conditionId) internal view {
    ...
    bytes32 collectionYes = ctfContract.getCollectionId(bytes32(0), conditionId, 1);
    bytes32 collectionNo  = ctfContract.getCollectionId(bytes32(0), conditionId, 2);
    uint256 expectedYes = ctfContract.getPositionId(collateralToken, collectionYes);
    uint256 expectedNo  = ctfContract.getPositionId(collateralToken, collectionNo);
    bool valid = ...;
    if (!valid) revert TokenConditionMismatch();
}
```

Proof of Concept

Admin registers a tokenId/complement pair for a conditionId that was never prepared — validation succeeds because the CTF view functions are pure.

Impact

False assurance from the NatSpec. Unprepared or non-binary conditions can slip through registration and produce surprising downstream behavior.

Recommendation

Add `require(ctf.getOutcomeSlotCount(conditionId) == 2)` to reject non-binary or unprepared conditions, and update the NatSpec to match actual behavior.

Status

Resolved

[L-05] Auth vs Resolution — asymmetric privilege-transfer safety

Description

The `Auth` mixin exposes only `addAdmin` / `removeAdmin` / `renounceAdminRole` with no two-step transfer, while `Resolution` inherits `Ownable2Step`. The higher-risk exchange admin role has weaker transfer safety than the `Resolution` owner role.

Vulnerability Details

```
// Auth.sol — single-step
function addAdmin(address admin_) external onlyAdmin { admins[admin_] = 1; ... }

// Resolution.sol — two-step via Ownable2Step
contract Resolution is Ownable2Step { ... }
```

Proof of Concept

Admin typos or delegates a role to a wrong address, then discovers the old admin is already removed; no pending/accept flow exists to recover.

Impact

Increased risk of accidental admin loss or social-engineering attacks on the higher-privilege role.

Recommendation

Implement a two-step admin transfer in `Auth` mirroring the `Ownable2Step` pattern.

Status

Resolved

QA

[Q-01] Trading — FeeCharged event not emitted in _fillOrder

Description

`_fillOrder` collects fees implicitly by paying the maker `taking - fee`, so the operator keeps the difference. Unlike `_chargeFee` (which emits `FeeCharged`), this flow emits no fee event.

Vulnerability Details

```
// Trading.sol:93-108 — _fillOrder
uint256 fee = CalculatorHelper.calculateFee(...);
_transfer(msg.sender, order.maker, takerAssetId, taking - fee);
_transfer(order.maker, to, makerAssetId, making);
// NOTE: Fees are "collected" by the Operator implicitly — no FeeCharged event
emit OrderFilled(...);
```

Proof of Concept

Operator accounting tools that consume `FeeCharged` events see fees from `matchOrders` and `_fillFacingExchange` but not from `fillOrder`, producing under-counted totals.

Impact

Off-chain accounting drift; no direct fund-loss consequence.

Recommendation

Emit a `FeeCharged` (or equivalent) event in `_fillOrder` whenever a non-zero fee is retained.

Status

Resolved

[Q-02] Operator trust model is not documented in-contract

Description

The operator has exclusive visibility of the off-chain order book and sole authority to submit `fillOrder`, `fillOrders`, and `matchOrders`. This enables selective execution, adverse-selection, and operator-chosen match-pair selection. These properties are inherited from the upstream Polymarket design but are not documented in the repo's NatSpec or README.

Vulnerability Details

```
// ProphetCTFExchange.sol
function fillOrder(Order memory order, uint256 fillAmount) external nonReentrant
onlyOperator notPaused { ... }
function fillOrders(...) external nonReentrant onlyOperator notPaused { ... }
function matchOrders(...) external nonReentrant onlyOperator notPaused { ... }
```

Proof of Concept

A user cannot determine from contract source alone that the operator is the sole submission path and has full discretion over which crossing orders to match.

Impact

Users may misunderstand the trust model. No direct fund-loss from this QA gap, but it amplifies the impact of other operator-centric findings (H-04, M-03, M-04).

Recommendation

Document the operator trust assumption in the contract NatSpec and user-facing README, including the concrete capabilities it implies.

Status

Resolved

Centralisation

The Prophet project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Prophet project seems to have a sound and well-tested codebase; now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with Resolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au